Solution: Sort List

Problem Statement

Given a `head` of the linked list, return the list after sorting it in ascending order.

Examples

1. **Example 1**:
    - **Input**: `[3, 1, 2]`
    - **Expected Output**: `[1, 2, 3]`
    - **Justification**: The list is sorted in ascending order, with `1` coming before `2`, and `2` before `3`.
2. **Example 2**:
    - **Input**: `[4]`
    - **Expected Output**: `[4]`
    - **Justification**: A list with a single element is already sorted.
3. **Example 3**:
    - **Input**: `[9, 8, 7, 6, 5, 4, 3, 2, 1]`
    - **Expected Output**: `[1, 2, 3, 4, 5, 6, 7, 8, 9]`
    - **Justification**: The list is sorted in ascending order, with each element being smaller than the next.

Solution

To sort a linked list, a merge sort algorithm can be used, which is renowned for its efficiency with linked lists due to its recursive nature and the ability to sort lists without additional space requirements for arrays. The approach involves dividing the list into smaller sublists until each sublist contains a single element or is empty. This division is based on the principle of recursion, where the list is split into halves repeatedly.

Once we have these small sublists, the next step is to merge them back together in a sorted manner. During the merging process, we compare the elements of the sublists and arrange them in ascending order. This merging is done until we reconstruct the entire list, but now in a sorted order.

Step-by-step algorithm

1. **Divide the List into Two Halves**:
    - **Step 1.1**: Start with a method that takes the head of the list as its parameter.
    - **Step 1.2**: If the list is empty or has only one node, it is already sorted, so return it as is.
    - **Step 1.3**: Use two pointers, slow and fast, to find the middle of the list. The slow pointer advances one node at a time, while the fast pointer advances two nodes at a time. When the fast pointer reaches the end, the slow pointer will be at the midpoint.
    - **Step 1.4**: Break the list into two parts at the midpoint. The first part starts from the head to the slow pointer, and the second part starts from the next node to the slow pointer to the end of the list.
2. **Recursively Sort Each Half**:
    - **Step 2.1**: Recursively apply the same algorithm to the first half of the list.
    - **Step 2.2**: Do the same for the second half of the list.
    - **Step 2.3**: The recursion base case is a list with a single node or an empty list, which is considered sorted.
3. **Merge the Two Sorted Halves**:
    - **Step 3.1**: After the two halves are sorted, merge them into a single sorted list.
    - **Step 3.2**: Create a dummy node to help with merging.
    - **Step 3.3**: Compare the heads of the two halves. Whichever node has the smaller value becomes the next node of the merged list.
    - **Step 3.4**: Continue this process until all nodes from both halves are merged.
    - **Step 3.5**: If one half is exhausted before the other, append the remaining nodes of the other half to the merged list.
4. **Return the Sorted List**:
    - **Step 4.1**: The head of the merged list (starting from the next node of the dummy node) is the head of the sorted list.
    - **Step 4.2**: Return the sorted list.

Algorithm Walkthrough

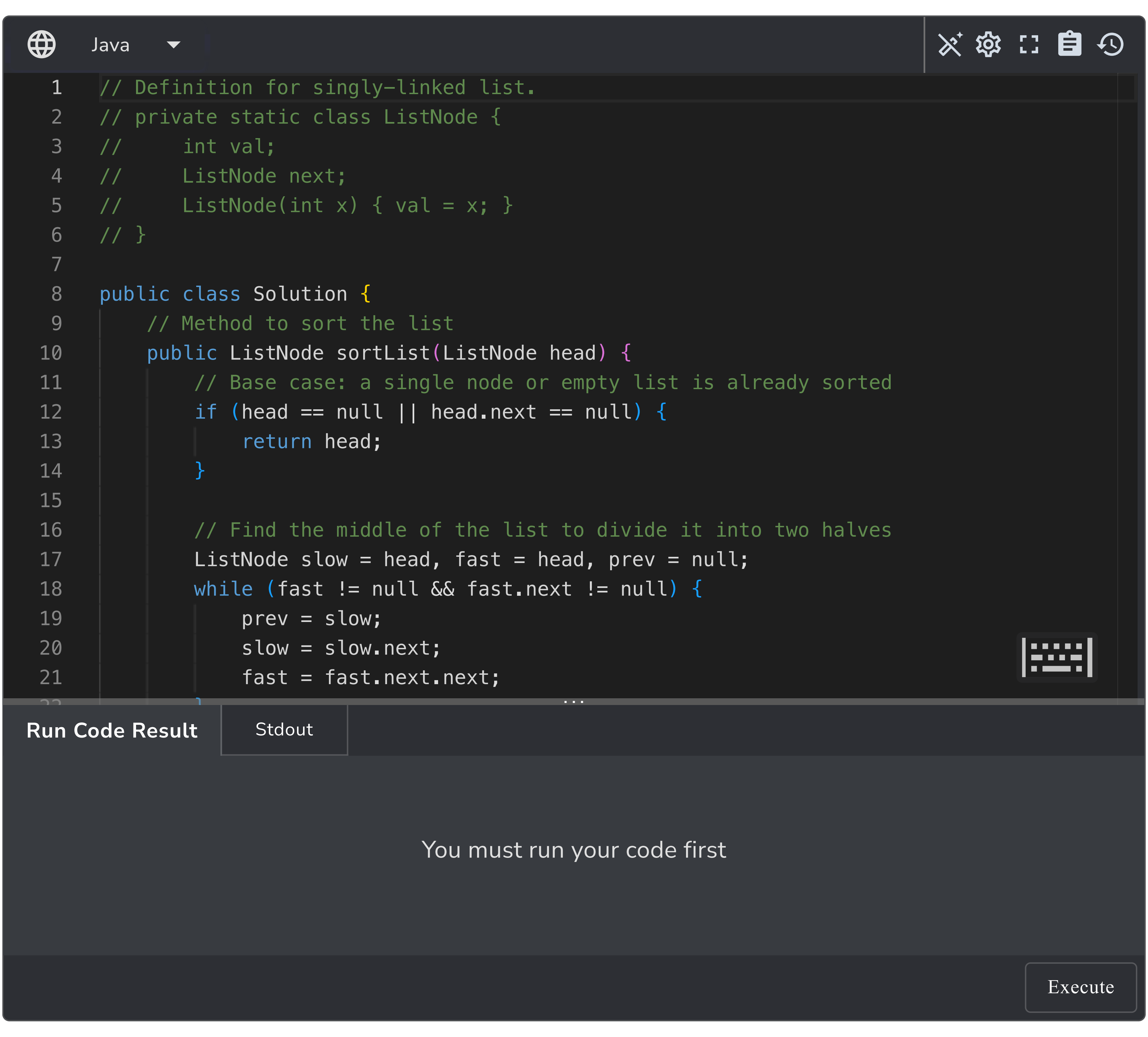
Given the input list `[9, 8, 7, 6, 5, 4, 3, 2, 1]`:

1. **Initial Division**:
    - Divide the list into two halves: `[9, 8, 7, 6, 5]` and `[4, 3, 2, 1]`.
2. **Further Division of First Half**:
    - Divide `[9, 8, 7, 6, 5]` into `[9, 8, 7]` and `[6, 5]`.
    - Continue dividing:
        - `[9, 8, 7]` becomes `[9, 8]` and `[7]`.
        - `[6, 5]` becomes `[6]` and `[5]`.
    - Keep dividing until single elements:
        - `[9, 8]` becomes `[9]` and `[8]`.
3. **Further Division of Second Half**:
    - Similarly, divide `[4, 3, 2, 1]` into `[4, 3]` and `[2, 1]`.
    - Continue dividing:
        - `[4, 3]` becomes `[4]` and `[3]`.
        - `[2, 1]` becomes `[2]` and `[1]`.
4. **Merging Sublists**:
    - Start merging:
        - Merge `[9]` and `[8]` into `[8, 9]`.
        - Merge `[8, 9]` and `[7]` into `[7, 8, 9]`.
        - Merge `[6]` and `[5]` into `[5, 6]`.
        - Merge `[7, 8, 9]` and `[5, 6]` into `[5, 6, 7, 8, 9]`.
    - For the second half:
        - Merge `[4]` and `[3]` into `[3, 4]`.
        - Merge `[2]` and `[1]` into `[1, 2]`.
        - Merge `[3, 4]` and `[1, 2]` into `[1, 2, 3, 4]`.
5. **Final Merge**:
    - Merge the two halves `[5, 6, 7, 8, 9]` and `[1, 2, 3, 4]` into `[1, 2, 3, 4, 5, 6, 7, 8, 9]`.

Each step involves either dividing the list into smaller sublists or merging already sorted sublists, ensuring that the final list is sorted in ascending order.

Code

Here is the code for this algorithm:



```java
// Definition for singly-linked list.
// private static class ListNode {
//     int val;
//     ListNode next;
//     ListNode(int x) { val = x; }
// }

public class Solution {
    // Method to sort the list
    public ListNode sortList(ListNode head) {
        // Base case: a single node or empty list is already sorted
        if (head == null || head.next == null) {
            return head;
        }

        // Find the middle of the list to divide it into two halves
        ListNode slow = head, fast = head, prev = null;
        while (fast != null && fast.next != null) {
            prev = slow;
            slow = slow.next;
            fast = fast.next.next;
```

You must run your code first

Time and Space Complexity

- **Time Complexity**: (O(n log n))
    - The algorithm recursively splits the list into halves (logarithmic number of splits) and then merges them (linear time for each merge), resulting in (O(n \log n)) time complexity.
- **Space Complexity**: (O(log n))
    - The space complexity is dominated by the recursive call stack, which grows proportionally to the height of the recursion tree, resulting in (O(\log n)) space complexity.